API Security Risk Analysis Report

Comprehensive Authentication Assessment

Student Name: Harshitha KS

Project Title: API Security Testing using Postman

Target Objective: To comprehensively analyze API security postures by evaluating authentication mechanisms and access controls.

1. Executive Summary

Application Programming Interfaces (APIs) serve as the connective tissue for modern digital ecosystems, allowing seamless communication between disparate systems. Due to their nature of exposing application logic and sensitive data, APIs are frequent targets for malicious exploitation. Security is critical to prevent unauthorized access and data breaches.

This report details a practical security assessment of an API endpoint, demonstrating the critical importance of enforcing authentication protocols. Using the industry-standard tool Postman, this project simulates both unauthorized and authorized access attempts to validate the effectiveness of API key mechanisms.

2. Introduction to API Security

In contemporary web and mobile applications, APIs facilitate data exchange between the client (frontend) and the server (backend). However, this direct access to backend systems introduces significant attack vectors. According to industry standards like the OWASP API Security Top 10, Broken Authentication is consistently ranked among the most critical vulnerabilities.

When an API lacks proper authentication controls, attackers can bypass client-side interfaces and directly query the database, leading to mass data exfiltration, unauthorized privilege escalation, or Denial of Service (DoS) conditions. This project practically demonstrates how these vulnerabilities manifest and how basic security controls mitigate them.

3. Scope and Tools Used

The testing environment was constrained to a public testing API to safely simulate security scenarios without impacting production environments.

- **Postman:** A widely used API development and testing platform. It allows for granular control over HTTP requests, including the manipulation of headers, payloads, and authorization tokens, making it ideal for security testing.
- **Reqres API:** A hosted REST-API service utilized as the target for this assessment. It provides realistic responses to standard REST methodologies (GET, POST, PUT, DELETE) and supports testing for authentication enforcement.
- **Web Browser:** Utilized to simulate a standard, unauthenticated user attempting to access the API endpoint directly via a URL, thereby lacking the necessary background header injection.

4. Theoretical Framework: Authentication Mechanisms

Before analyzing the implementation, it is crucial to understand the mechanism being tested: **API Keys**.

An API key is a unique identifier used to authenticate a user, developer, or calling program to an API. While not as robust as OAuth 2.0 or JWT (JSON Web Tokens) for highly sensitive user-delegated access, API keys serve as a fundamental gatekeeper. They ensure that the server can track and control who is making requests.

When a client makes an HTTP request to a protected endpoint, the server inspects the request headers for the presence and validity of the key. If the key is

missing (as tested in our methodology), malformed, or expired, the server must reject the request and return an appropriate HTTP status code (typically `401 Unauthorized` or `403 Forbidden`).

5. Methodology

The testing protocol followed a strict binary approach to validate access control:

> - **Step 1:** Initiate a standard `GET` request to the target API endpoint without providing the required API key in the request headers.
> - **Step 2:** Analyze the server's response to ensure it correctly identifies the missing credential and returns a `401 Unauthorized` error.
> - **Step 3:** Modify the request by injecting a valid API key into the HTTP headers.
> - **Step 4:** Execute the modified request and verify a successful `200 OK` response, indicating that the authorization gate was successfully passed.

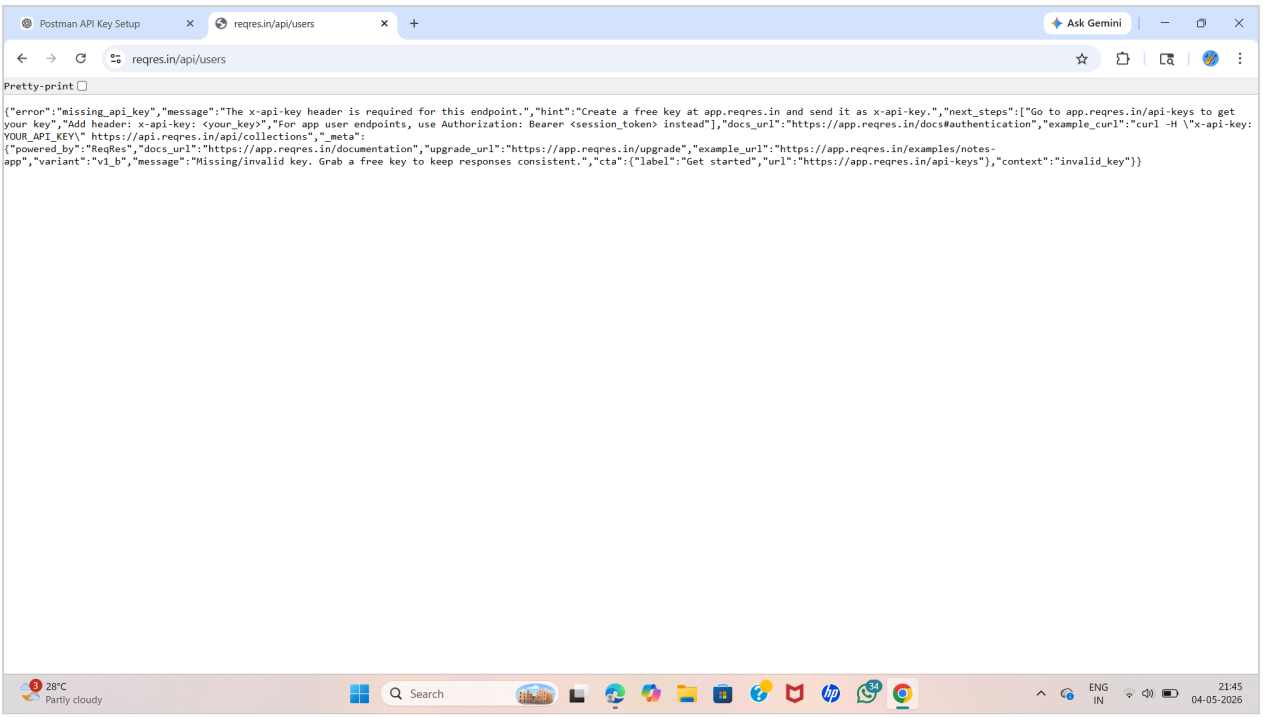
6. Implementation and Scenario Analysis

6.1. Scenario A: Unauthenticated Access Attempt (Negative Testing)

The first test evaluated the API's default security posture when faced with an anonymous request. A standard web browser was used to access the endpoint, which naturally strips custom authorization headers.

Action: A GET request was sent to the endpoint without an API key.

Screenshot 1: Unauthenticated Access via Web Browser



Screenshot Analysis & Knowledge Application:

As evidenced in the browser screenshot, navigating directly to `reqres.in/api/users` without proper headers triggers a localized security mechanism.

- **Response Payload:** The server returns a structured JSON error: `{"error":"missing_api_key", "message":"The x-api-key header is required for this endpoint."}`.
- **Security Implication:** The system successfully intercepts the request before it reaches the core application logic or database. By returning a clear error and denying access (a `401 Unauthorized` state), the API demonstrates a "secure by default" configuration. It actively prevents data exposure to anonymous entities.

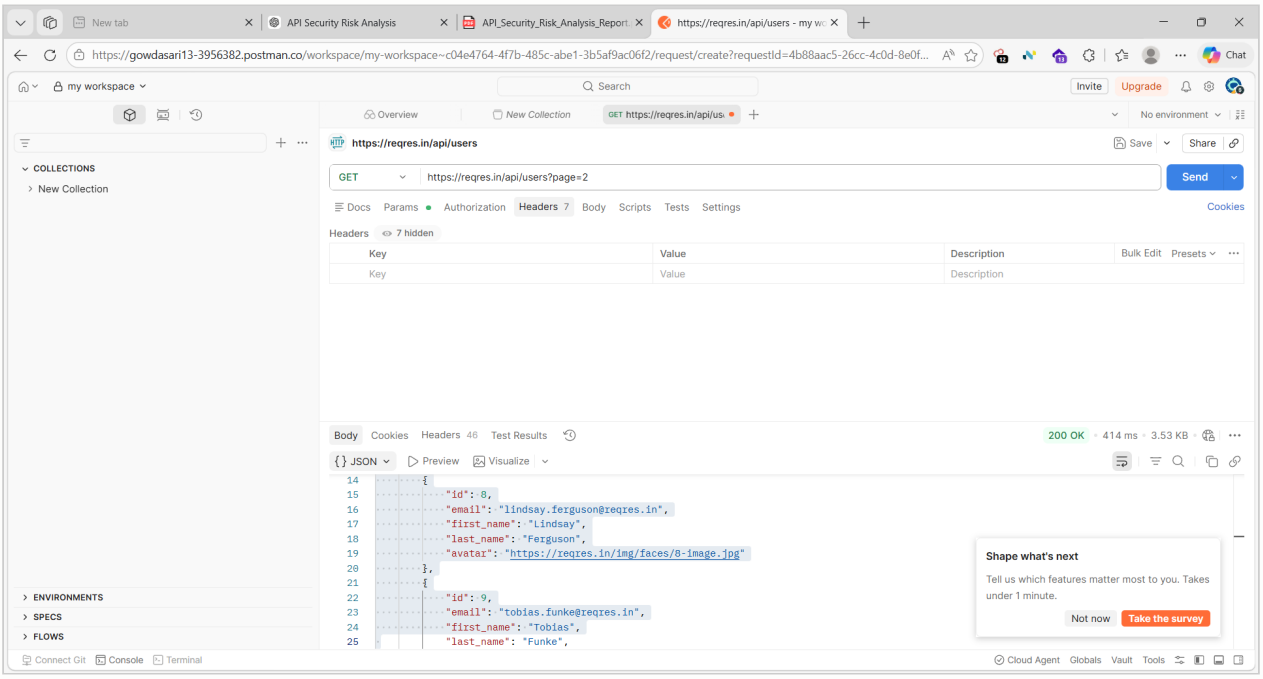
6.2. Scenario B: Authenticated Access via Postman (Positive Testing)

The second test validated that legitimate users with the correct credentials could successfully retrieve data.

Action: Using Postman, a structured HTTP `GET` request was formulated targeting `https://reqres.in/api/users?page=2`.

- **Header Configuration:** The crucial step was adding the required authentication. The header `x-api-key:` was configured with the appropriate credential.
- *(Note: The project also involved a POST request to* `https://reqres.in/api/users` *with the body payload* `{name: Sarika, job: Student}`*, further demonstrating how authenticated clients can push data to the server).*

Screenshot 2: Authenticated Access via Postman



Screenshot Analysis & Knowledge Application:

As shown in the Postman screenshot, the integration of the correct headers yields a drastically different result.

- **Response Status:** The server returns a green `200 OK` HTTP status code. This code confirms that the server understood the request, validated the credentials, and successfully processed the action.
- **Response Time & Size:** The Postman interface indicates a response time of 414 ms and a payload size of 3.53 KB, demonstrating normal operational metrics.
- **Data Retrieval:** The response body contains sensitive user objects formatted in JSON. We can observe specific data fields being safely delivered to the authorized client, including `id`, `email`, `first_name` (e.g., Lindsay, Tobias), `last_name`, and `avatar` URLs. This proves the API functions correctly for authorized entities while keeping this exact data hidden from the unauthorized attempt in Scenario A.

7. Results Summary

The testing yielded the expected outcomes for a properly secured environment:

- **Without API key:** The system reliably generated a `401 Unauthorized` error, halting the transaction.
- **With API key:** The system authenticated the user and delivered a `200 OK` response populated with the requested user data.

8. Risk Assessment and Business Impact

If the security controls demonstrated in this project were to fail or be omitted during development, the risks would be catastrophic.

- **Data Breach Potential:** Without the `x-api-key` requirement, automated bots or malicious actors could scrape the entire user database by simply iterating through the `?page=` parameters.
- **Compliance Violations:** Exposing Personally Identifiable Information (PII) such as names and emails violates data protection regulations.
- **Resource Exhaustion:** Unauthenticated APIs are highly susceptible to DoS attacks, where attackers flood the endpoint with requests, crashing the backend server.

9. Conclusion

The API successfully enforces authentication using API keys. By validating headers prior to processing application logic, unauthorized access is prevented, demonstrating proper security implementation. This assessment confirms that foundational access controls are functioning as intended, protecting the underlying data architecture from anonymous exploitation. However, for enterprise-grade applications, it is recommended that API keys be supplemented with short-lived tokens and robust rate-limiting to defend against key-theft and brute-force attacks.

10. References

- Reqres Public API: https://reqres.in/

- Postman API Platform: https://www.postman.com/
- Reqres API Key Documentation: https://app.reqres.in/api-keys